

Кэширование. Стратегии и реализации

ВПВ по курсу “Алгоритмы и структуры данных”

Бусарова Галина Васильевна Б01-411 11 декабря 2025

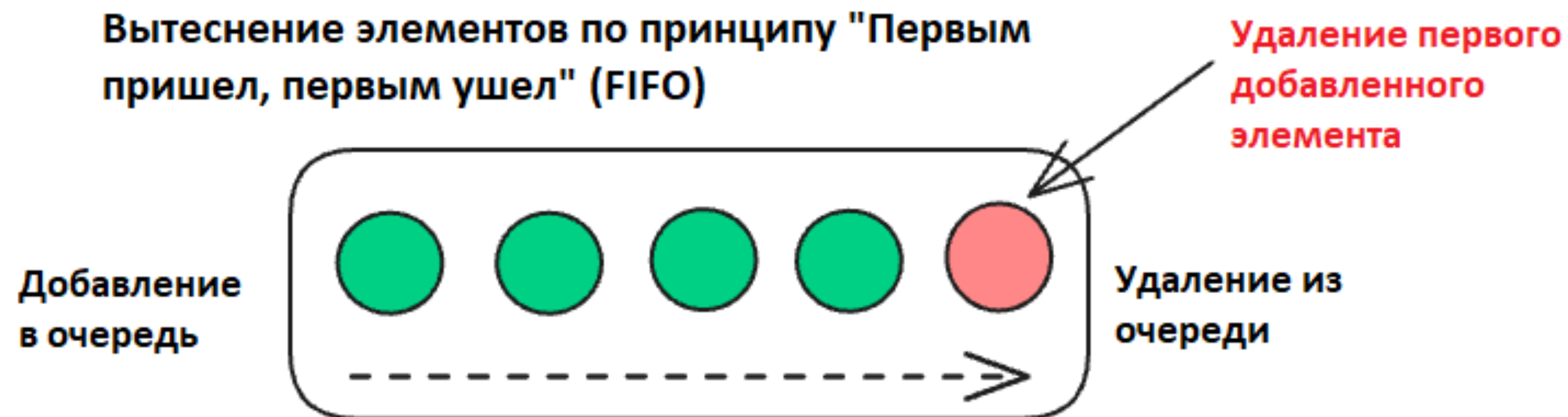
Что такое кэширование и для чего оно нужно?

- **Кэш** - это быстрая, но небольшая память, куда кладут часто используемые данные, чтобы не ходить каждый раз в медленное, но большое хранилище.
- Кэш маленький (например, на 1000 элементов), а данных - миллионы. Нужно решать что хранить, а что выкидывать.
- Это и делают стратегии кэширования: LRU, LFU и другие.

Популярные стратегии кэширования

FIFO - <First In, First Out>

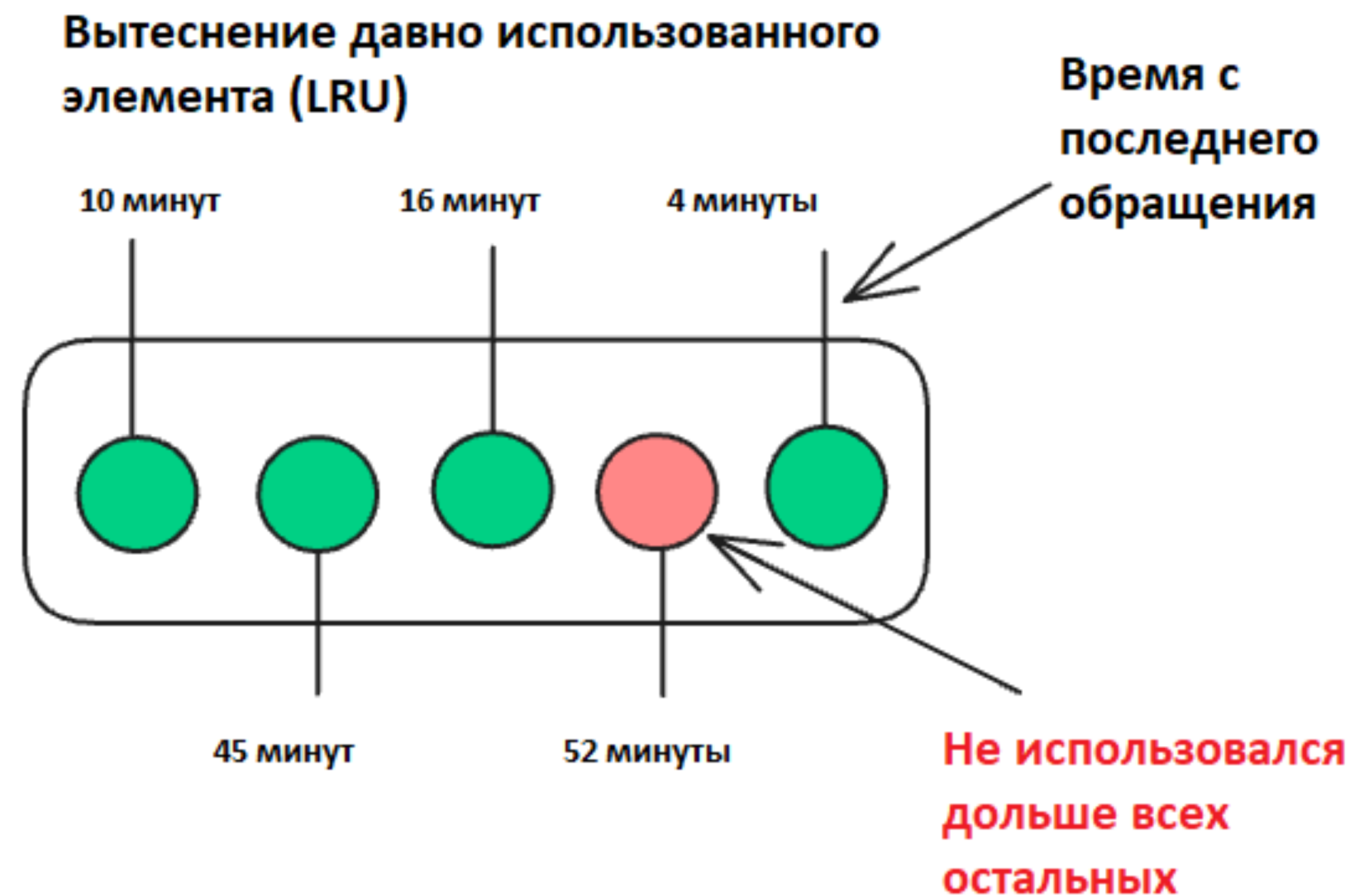
- **Принцип:** выкидываем самого старого по времени добавления



Популярные стратегии кэширования

LRU - <Least Recently Used>

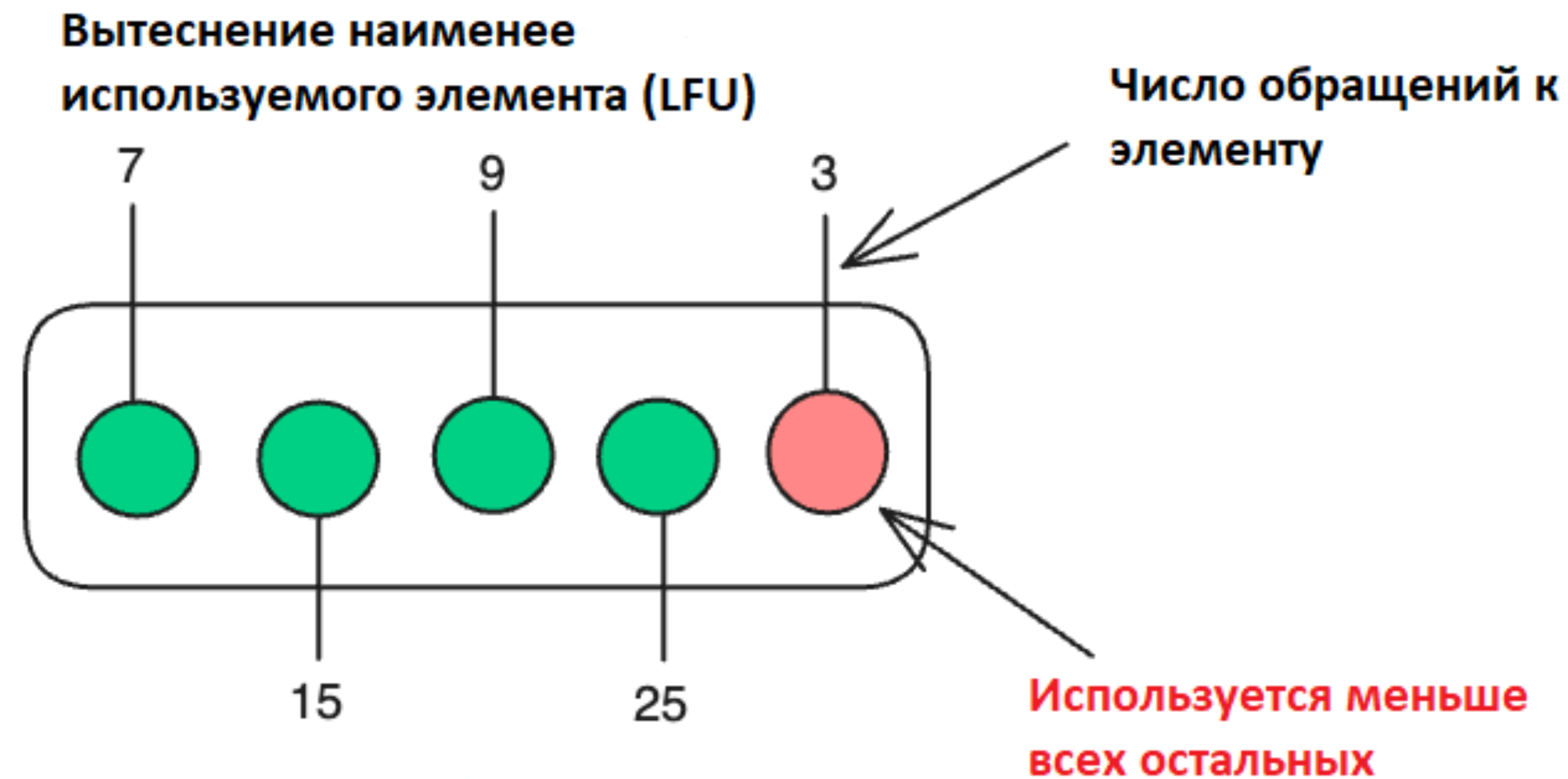
- Принцип: выкидываем самого давно использованного



Популярные стратегии кэширования

LFU - <Least Frequently Used>

- Принцип: выкидываем с минимальной частотой.



Актуальность

Зачем это нужно?

- Реальная задача: веб-сервер кэширует профили пользователей: БД: 10 мс/запрос, Кэш: 0.1 мкс/запрос, Нагрузка: 50 000 RPS
- Даже 1 % miss'ов = 500 запросов/сек в БД -> тайм-аут и падение сервиса.
- Выбор алгоритма кэширования - вопрос живучести системы.

Модель замеров

Trace-based benchmarking - стандарт в индустрии

- Вход: последовательность запросов("trace")
- Выход: **hit rate** = % запросов, найденных в кэше

Моя модель:

- 100 000 запросов
- 80% — Zipf($\alpha=1.1$): 5% ключей = 60% запросов
- 10% — *scan-атака*: 1, 2, 3, ..., 10 000 (одноразовые)
- 10% — повторы «горячих» ключей (10, 20, 30, 40)

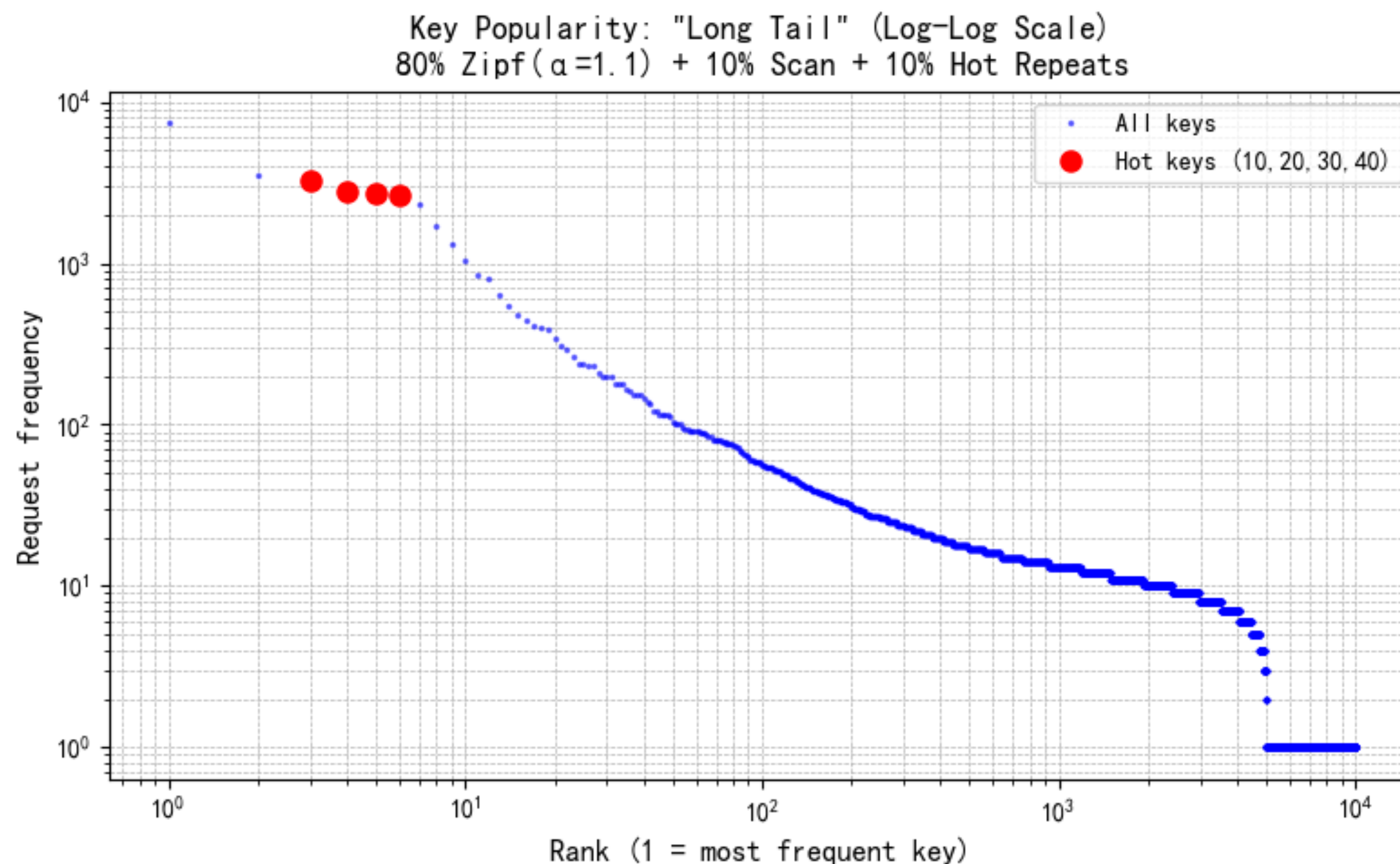
Цель: проверить устойчивость к реальным нагрузкам

Гистограмма популярности ключей

— Синяя кривая — Zipf($\alpha=1.1$): «длинный хвост» (как в Википедии и соцсетях),

— Красные точки — «горячие» ключи, дополнительно запрашиваемые 10% времени,

— Точки у оси X (частота = 1) — scan-атака: 10 000 одноразовых ключей.



Именно такая смесь делает нагрузку реалистичной и проверяет устойчивость кэшей.

Математическая формулировка

Дано:

- поток запросов $\sigma = (x_1, \dots, x_n)$, где $x_i \in U$
- кэш емкости $k \ll |U|$

На шаге i :

- если $x_i \in \text{cache} \rightarrow \text{hit}$
- иначе - miss

Цель: максимизировать hit rate.

LRU - Least Recently Used

- Реализация: двусвязный список + хэш-таблица -> O(1) get/put
- Плюсы:
- Отлично ловит временную локальность (A, B, C -> A снова = hit)
- Минусы:
- Уязвим к scan-атаке

```
request = get_next_request()
while request != END_OF_REQUESTS do
    // Проверяем кэш
    cache_entry = cache.get(request)

    if cache_entry != nil then
        // HIT: запрос в кэше
        hits = hits + 1

        // Обновляем порядок (перемещаем в начало)
        list.remove(cache_entry.node)
        list.add_to_front(cache_entry.node)

        result = cache_entry.data
    else
        // MISS: запроса нет в кэше
        misses = misses + 1

        // Если кэш полный, удаляем старый запрос
        if cache.size() >= capacity then
            lru_node = list.remove_last()
            cache.remove(lru_node.request)
        end if

        // Сохраняем в кэш
        new_node = new Node(request)
        list.add_to_front(new_node)
        cache_entry = new CacheEntry(result, new_node)
        cache.put(request, cache_entry)
    end if

    // Отдаем результат
    send_response(result)

    // Следующий запрос
    request = get_next_request()
end while
```

LFU - Least Frequently Used

- Реализация: хэш-таблица, двусвязный список(для каждой частоты свой - от самого нового запроса к самому старому)
- **Плюсы:**
- Устойчив к сканам
- **Минусы:**
- Частоты растут -> переполнение

BENCHMARK: Как меряли

- **Инструменты:**

- Генератор trace'ов (Python)
- C++17 реализации.

- **Данные:**

- 100K запросов
- 10K уникальных ключей

Требования:

- C++17 (GCC $\geq$ 7.0 / Clang $\geq$ 5.0)
- CMake $\geq$ 3.14

BENCHMARK: Как меряли

- Метрики:
- Hit rate (%)
- Latency (нс/операцию)

```
busarovvasily@Busarovs-MacBook-Pro LFU_cache % make full-benchmark
LRU быстрее LFU на: 4.0 мс (57.1% быстрее)

СРАВНЕНИЕ ПРИ РАЗНЫХ РАЗМЕРАХ КЭША:
-----
РазмерLRU Hit %      LFU Hit %      Преимущество LFU
-----
100      28.67      38.21      9.54      %
500      41.41      47.33      5.92      %
1000     47.93      52.14      4.20      %
2000     56.67      59.23      2.57      %
5000     73.65      74.54      0.89      %

Бенчмарк завершен!
```

Литература + репозиторий

- GitHub: https://github.com/GalyaBusarova/lfu-cache_project
- **Литература:**
- Cohen, Einziger. TinyLFU: A Highly Efficient Cache Admission Policy (2015)
- GeeksforGeeks — LFU Cache Implementation
- Wikipedia - Cache replacement policies
- **Спасибо за внимание!**